# `zip`


| | |
|---|---|
| Document #: | P2321R1 |
| Date: | 2021-04-11 |
| Project: | Programming Language C++ |
| Audience: | LEWG |
| | LWG |


| | |
|---|---|
| Reply-to: | Tim Song |
| | <t.canens.cpp@gmail.com> |


# Contents


- 1 Abstract
- 2 Revision history
- 3 Examples
- 4 Discussion
  - 4.1 Proxy reference changes
  - 4.2 `zip` and `zip_transform`
    - 4.2.1 No common exposition-only base view
    - 4.2.2 `tuple` or `pair`?
    - 4.2.3 Zipping nothing?
    - 4.2.4 When is `zip_view` a `common_range`?
  - 4.3 `adjacent` and `adjacent_transform`
    - 4.3.1 Naming
    - 4.3.2 Value type
    - 4.3.3 `common_range`
    - 4.3.4 No input ranges
    - 4.3.5 `iter_swap`
- 5 Wording
  - 5.1 `tuple`
  - 5.2 `pair`
  - 5.3 `vector<bool>::reference`
  - 5.4 Addition to `<ranges>`
  - 5.5 `zip`
    - 24.7.? Zip view [range.zip]
  - 5.6 `zip_transform`
    - 24.7.? Zip transform view [range.zip.transform]
  - 5.7 `adjacent`
    - 24.7.? Adjacent view [range.adjacent]
  - 5.8 `adjacent_transform`
    - 24.7.? Adjacent transform view [range.adjacent.transform]
  - 5.9 Feature-test macro
- 6 Acknowledgements

7 References

# 1 Abstract

This paper proposes

— four views, `zip`, `zip_transform`, `adjacent`, and `adjacent_transform`,

— changes to `tuple` and `pair` necessary to make them usable as proxy references (necessary for `zip` and `adjacent`), and

— changes to `vector<bool>::reference` to make it usable as a proxy reference for writing,

all as described in section 3.2 of [P2214R0].

# 2 Revision history

— R1: Added feature test macro. Expanded discussion regarding 1) `operator==` for forward-or-weaker `zip` iterators and 2) `adjacent` on input ranges. Miscellaneous wording fixes (thanks to Barry Revzin and Tomasz Kamiński). Added a short example.

# 3 Examples

```
std::vector v1 = {1, 2};
std::vector v2 = {'a', 'b', 'c'};
std::vector v3 = {3, 4, 5};

fmt::print("{}\n", std::views::zip(v1, v2));                                  // {(1,
        'a'), (2, 'b')}
fmt::print("{}\n", std::views::zip_transform(std::multiplies(), v1, v3));  // {3, 8}
fmt::print("{}\n", v2 | std::views::pairwise);                                // {('a',
        'b'), ('b', 'c')}
fmt::print("{}\n", v3 | std::views::pairwise_transform(std::plus()));      // {7, 9}
```

# 4 Discussion

The proposed wording below generally follows the design described in section 3.2 of [P2214R0], and the discussion in this paper assumes familiarity with that paper. This section focuses on deviations from and additions to that paper, as well as certain details in the design of the views that should be called out.

## 4.1 Proxy reference changes

The basic rationale for changes to `tuple` and `pair` are described in exhaustive detail in [P2214R0] sections 3.2.1 and 3.2.2 and will not be repeated here. Several additions are worth noting:

— `common_type` and `basic_common_reference` specializations are added for `tuple` and `pair`. These are also required for `tuple` and `pair` to be usable as proxy references.

— `swap` for `const tuple` and `const pair`. Once tuples of references are made const-assignable, the default `std::swap` can be called for const tuples of references. However, that triple-move `swap` does the wrong thing:

```
int i = 1, j = 2;
const auto t1 = std::tie(i), t2 = std::tie(j);

// If std::swap(t1, t2); called the default triple-move std::swap then
// this would do
auto tmp = std::move(t1);
t1 = std::move(t2);
t2 = std::move(tmp);

// i == 2, j == 2
```

This paper therefore proposes adding overloads of `swap` for `const` tuples and pairs to correctly perform element-wise swap.

— Consistent with the scoped allocator protocol, allocator-extended constructors that correspond to the new `tuple` constructors have been added to `tuple`, and new overloads of `uses_allocator_construction_args` corresponding to the new `pair` constructors have been added as well.

## § 4.2 `zip` and `zip_transform`

### § 4.2.1 No common exposition-only base view

[P2214R0] proposes implementing `zip` and `zip_transform` to produce specializations of an exposition-only *`iter-zip-transform-view`*, which is roughly how they are implemented in range-v3. In the process of writing wording for these views, however, it has become apparent that the two views have enough differences that a common underlying view would need to have additional knobs to control the behavior (beyond the `value_type` issue already noted in the paper). The extra complexity required would likely negate any potential benefit from having a single underlying view.

Instead, the wording below specifies `zip_transform` largely in terms of `zip`. This significantly reduces specification duplication without sacrificing efficiency.

### § 4.2.2 `tuple` or `pair`?

In range-v3, zipping two views produced a range of `pairs`, while zipping any other number of ranges produce a range of `tuples`. This paper maintains that design for several reasons:

— Since `pair` is tuple-like, most common uses of the result (`get`, `apply`, structured bindings, etc.) would work just as well.

— Certain parts of the standard library, notably `map` and friends, are dependent on `pair`.

— `pair` implicitly converts to `tuple` if one is really needed, whereas constructing a `pair` from a `tuple` is more difficult.

### § 4.2.3 Zipping nothing?

As in range-v3, zipping nothing produces an `empty_view` of the appropriate type.

### 4.2.4 When is `zip_view` a `common_range`?

A `common_range` is a range whose iterator and sentinel types are the same.

Obviously, when zipping a single range, the `zip_view` can be a `common_range` if the underlying range is.

When the `zip_view` is not bidirectional, it can be a `common_range` when every underlying view is a `common_range`. To handle differently-sized ranges, iterator `==` is a logical OR: two iterators compare equal if one of the sub-iterators compare equal. Note that the domain of `==` only extends to iterators over the same underlying sequence; the use of logical OR is valid within that domain because the only valid operands to `==` are iterators obtained from incrementing `begin()` zero or more times and the iterator returned by `end()`.

When the `zip_view` is bidirectional (or stronger), however, it is now possible to iterate backwards from the end iterator (if it is indeed an iterator). As a result, we cannot simply construct the end iterator out of the end iterators of the views: if the views are different in size, iterating backwards from the end will give us elements that are not in the view at all (see [range-v3.1592]). Instead, we need to produce a "proper" end iterator by advancing from `begin`; to be able compute `end` in constant time, we need all views to be random access and sized.

As `end` is only required to be amortized constant time, it is in theory possible to do a linear time traversal and cache the result. The additional benefit from such a design appears remote, and it has significant costs.

#### 4.2.4.1 Does that mean `views::enumerate(a_std_list)` isn't a `common_range`?

It's still an open question whether `enumerate` should be implemented in terms of `zip_view` or not. If it is specified in terms of `zip_view` (and produces a `pair`), as proposed in [P2214R0], it is easy to specify an separate `enumerate_view` that is implemented with `zip_view` but still produces a common range for this case.

#### 4.2.4.2 What about infinite ranges in general?

If `zip_view` can recognize when a range is infinite, then it is theoretically possible for it to be a `common_range` in the following two cases:

- Every range but one is an infinite random-access range, while the one finite range is a sized and common range (of any iterator strength). This is the general case of `enumerate` above.
- Every range is either random-access and sized, or random-access and infinite, and at least one range is sized.

The standard, however, generally does not recognize infinite ranges (despite providing `unreachable_sentinel`). It goes without saying that a complete design for infinite ranges support is outside the scope of this paper.

## § 4.3 `adjacent` and `adjacent_transform`

As `adjacent` is a specialized version of `zip`, most of the discussion in above applies, *mutatis mutandis*, to `adjacent` as well, and will not be repeated here.

### § 4.3.1 Naming

The wording below tentatively uses `adjacent` for the general functionality, and `pairwise` for the `N == 2` case. [P2214R0] section 3.2.5 suggests an alternative (`slide_as_tuple` for the general functionality and `adjacent` for the `N == 2` case). The author has a mild preference for the current names due to the somewhat unwieldiness of the name `slide_as_tuple`.

### § 4.3.2 Value type

The value type of `adjacent_view` is a homogeneous `tuple` or `pair`. Since `array` cannot hold references and is defined to be an aggregate, using it as the value type poses significant usability issues (even if we somehow get the `common_reference_with` requirements in `indirectly_readable` to work with even more `tuple`/`pair` changes).

### § 4.3.3 `common_range`

One notable difference from `zip` is that since `adjacent` comes from a single underlying view, it can be a `common_range` whenever its underlying view is.

### § 4.3.4 No input ranges

Because `adjacent` by definition holds multiple iterators to the same view, it requires forward ranges. It is true that the `N == 1` case could theoretically support input ranges, but that adds extra complexity and seems entirely pointless. Besides, someone desperate to wrap their input range in a single element `tuple` can just use `zip` instead.

During LEWG review of R0 of this paper it was suggested that `adjacent<N>` could support input views by caching the elements referred to by the last `N` iterators. Such a view would have significant differences from what is being proposed in this paper. For instance, because the reference obtained from an input iterator is invalidated on increment, the range will have to cache by value type, and so the reference type will have to be something like `tuple<range_value_t<V>&...>` (or perhaps even `tuple<range_value_t<V>...>&`?) instead of `tuple<range_reference_t<V>...>`. To be able to construct and update the cached values, the view would have to require underlying range's value type to be constructible and assignable from its reference type. And because we don't know what elements the user may desire to access, iterating through the view necessarily requires copying every element of the underlying view into the cache, which can be wasteful if not all elements need to be accessed. By comparison, iterating through the proposed `adjacent` copies exactly zero of the underlying range's elements.

Additionally, because input views provide much fewer operations and guarantees, they can often be implemented more efficiently than forward views. There has been an open range-v3 issue [range-v3.704] since 2017 (see also this comment from Eric Niebler on /r/cpp) to provide an API that downgrades a

forward-or-stronger range to input for efficiency when the forward range's guarantees are not needed. Having a view adaptor that is significantly more expensive when given an input range would significantly damage the usability and teachability of such a design.

The author believes that the behavioral and performance characteristics of such a view is different enough from the `adjacent` proposed in this paper that it would be inappropriate to put them under the same name. It can be proposed separately if desired.

### 4.3.5 `iter_swap`

Since the iterators of `adjacent_view` refer to potentially overlapping elements of the underlying view, `iter_swap` cannot really "exchange the values" of the range elements when the iterators overlap. However, it does not appear to be possible to disable `ranges::iter_swap` (deleting or not providing `iter_swap` will simply fallback to the default implementation), and swapping non-overlapping iterators is still useful functionality. Thus, the wording below retains `iter_swap` but gives it a precondition that there is no overlap.

# 5 Wording

This wording is relative to [N4878].

## 5.1 `tuple`

1. Edit 20.5.2 [tuple.syn], header `<tuple>` synopsis, as indicated:

```
#include <compare>              // see [compare.syn]

namespace std {
  // [tuple.tuple], class template tuple
  template<class... Types>
    class tuple;

+  template<class... TTypes, class... UTypes, template<class> class TQual,
       template<class> class UQual>
+    requires requires { typename tuple<common_reference_t<TQual<TTypes>,
       UQual<UTypes>>...>; }
+  struct basic_common_reference<tuple<TTypes...>, tuple<UTypes...>, TQual,
       UQual> {
+    using type = tuple<common_reference_t<TQual<TTypes>,
       UQual<UTypes>>...>;
+  };
+
+  template<class... TTypes, class... UTypes>
+    requires requires { typename tuple<common_type_t<TTypes, UTypes>...>; }
+  struct common_type<tuple<TTypes...>, tuple<UTypes...>> {
+    using type = tuple<common_type_t<TTypes, UTypes>...>;
+  };


  // [...]
```

```
  // [tuple.special], specialized algorithms
  template<class... Types>
    constexpr void swap(tuple<Types...>& x, tuple<Types...>& y)
       noexcept(see below);
+ template<class... Types>
+   constexpr void swap(const tuple<Types...>& x, const tuple<Types...>& y)
       noexcept(see below);
}
```

2. Edit 20.5.3 [tuple.tuple], class template `tuple` synopsis, as indicated:

```
namespace std {
  template<class... Types>
  class tuple {
  public:
    // [tuple.cnstr], tuple construction
    constexpr explicit(see below) tuple();
    constexpr explicit(see below) tuple(const Types&...);         // only
       if sizeof...(Types) >= 1
    template<class... UTypes>
      constexpr explicit(see below) tuple(UTypes&&...);           // only
       if sizeof...(Types) >= 1

    tuple(const tuple&) = default;
    tuple(tuple&&) = default;

+   template<class... UTypes>
+     constexpr explicit(see below) tuple(tuple<UTypes...>&);
    template<class... UTypes>
      constexpr explicit(see below) tuple(const tuple<UTypes...>&);
    template<class... UTypes>
      constexpr explicit(see below) tuple(tuple<UTypes...>&&);
+   template<class... UTypes>
+     constexpr explicit(see below) tuple(const tuple<UTypes...>&&);

+   template<class U1, class U2>
+     constexpr explicit(see below) tuple(pair<U1, U2>&);          // only
       if sizeof...(Types) == 2
    template<class U1, class U2>
      constexpr explicit(see below) tuple(const pair<U1, U2>&);   // only
       if sizeof...(Types) == 2
    template<class U1, class U2>
      constexpr explicit(see below) tuple(pair<U1, U2>&&);        // only
       if sizeof...(Types) == 2
+   template<class U1, class U2>
+     constexpr explicit(see below) tuple(const pair<U1, U2>&&);  // only
       if sizeof...(Types) == 2

    // allocator-extended constructors
    template<class Alloc>
      constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a);
    template<class Alloc>
      constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, const Types&...);
```

```
    template<class Alloc, class... UTypes>
      constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, UTypes&&...);
    template<class Alloc>
      constexpr tuple(allocator_arg_t, const Alloc& a, const tuple&);
    template<class Alloc>
      constexpr tuple(allocator_arg_t, const Alloc& a, tuple&&);
+   template<class Alloc, class... UTypes>
+     constexpr explicit(see below)
+       tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&);
    template<class Alloc, class... UTypes>
      constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&);
    template<class Alloc, class... UTypes>
      constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&&);
+   template<class Alloc, class... UTypes>
+     constexpr explicit(see below)
+       tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&&);
+   template<class Alloc, class U1, class U2>
+     constexpr explicit(see below)
+       tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&);
    template<class Alloc, class U1, class U2>
      constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&);
    template<class Alloc, class U1, class U2>
      constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);
+   template<class Alloc, class U1, class U2>
+     constexpr explicit(see below)
+       tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&&);

    // [tuple.assign], tuple assignment
    constexpr tuple& operator=(const tuple&);
+   constexpr const tuple& operator=(const tuple&) const;
    constexpr tuple& operator=(tuple&&) noexcept(see below);
+   constexpr const tuple& operator=(tuple&&) const;

    template<class... UTypes>
      constexpr tuple& operator=(const tuple<UTypes...>&);
+   template<class... UTypes>
+     constexpr const tuple& operator=(const tuple<UTypes...>&) const;
    template<class... UTypes>
      constexpr tuple& operator=(tuple<UTypes...>&&);
+   template<class... UTypes>
+     constexpr const tuple& operator=(tuple<UTypes...>&&) const;

    template<class U1, class U2>
      constexpr tuple& operator=(const pair<U1, U2>&);          // only if
       sizeof...(Types) == 2
+   template<class U1, class U2>
+     constexpr const tuple& operator=(const pair<U1, U2>&) const;    //
       only if sizeof...(Types) == 2
    template<class U1, class U2>
      constexpr tuple& operator=(pair<U1, U2>&&);               // only if
       sizeof...(Types) == 2
```

```
+    template<class U1, class U2>
+      constexpr const tuple& operator=(pair<U1, U2>&&) const;          //
        only if sizeof...(Types) == 2

     // [tuple.swap], tuple swap
     constexpr void swap(tuple&) noexcept(see below);
+    constexpr void swap(const tuple&) const noexcept(see below);
   };

   // [...]
 }
```

3. Edit 20.5.3.1 [tuple.cnstr] as indicated:

```
template<class... UTypes> constexpr explicit(see below)
        tuple(tuple<UTypes...>& u);
template<class... UTypes> constexpr explicit(see below) tuple(const
        tuple<UTypes...>& u);
template<class... UTypes> constexpr explicit(see below)
        tuple(tuple<UTypes...>&& u);
template<class... UTypes> constexpr explicit(see below) tuple(const
        tuple<UTypes...>&& u);
```

? Let `I` be the pack `0, 1, ..., (sizeof...(Types) - 1)`. Let `FWD(u)` be `static_cast<decltype(u)>(u)`.

? *Constraints*:

(?.1) — `sizeof...(Types)` equals `sizeof...(UTypes)`, and

(?.1) — `(is_constructible_v<Types, decltype(get<I>(FWD(u)))> && ...)` is `true`, and

— either `sizeof...(Types)` is not `1`, or (when `Types...` expands to `T` and `UTypes...` expands to `U`) `is_convertible_v<decltype(u), T>`, `is_constructible_v<T, decltype(u)>`, and `is_same_v<T, U>` are all false.

? *Effects:* For all *i*, initializes the $i^{\text{th}}$ element of `*this` with `get<i>(FWD(u))`.

? *Remarks:* The expression inside `explicit` is equivalent to:
`!conjunction_v<is_convertible<decltype(get<I>(FWD(u))), Types>...>`

```
template<class... UTypes> constexpr explicit(see below) tuple(pair<U1, U2>&
        u);
template<class... UTypes> constexpr explicit(see below) tuple(const pair<U1,
        U2>& u);
template<class... UTypes> constexpr explicit(see below) tuple(pair<U1, U2>&&
        u);
template<class... UTypes> constexpr explicit(see below) tuple(const pair<U1,
        U2>&& u);
```

? Let `FWD(u)` be `static_cast<decltype(u)>(u)`.

? *Constraints*:

(?.1) — `sizeof...(Types)` is 2 and

(?.2) — `is_constructible_v<T0, decltype(get<0>(FWD(u)))>` is `true` and

(?.2) — `is_constructible_v<T1, decltype(get<1>(FWD(u)))>` is `true`.

? *Effects:* Initializes the first element with `get<0>(FWD(u))` and the second element with `get<1>(FWD(u))`.

? *Remarks:* The expression inside `explicit` is equivalent to:
`!is_convertible_v<decltype(get<0>(FWD(u))), T0> ||`
`!is_convertible_v<decltype(get<1>(FWD(u))), T1>`

~~`template<class... UTypes> constexpr explicit(see below) tuple(const tuple<UTypes...>& u);`~~

18 ~~*Constraints:*~~

(18.1) — ~~`sizeof...(Types)` equals `sizeof...(UTypes)`, and~~

(18.2) — ~~`is_constructible_v<Ti, const Ui&>` is `true` for all *i*, and~~

(18.3) — ~~either `sizeof...(Types)` is not 1, or (when `Types...` expands to `T` and `UTypes...` expands to `U`) `is_convertible_v<const tuple<U>&, T>`, `is_constructible_v<T, const tuple<U>&>`, and `is_same_v<T, U>` are all `false`.~~

19 ~~*Effects:* Initializes each element of `*this` with the corresponding element of `u`.~~

20 ~~*Remarks:* The expression inside `explicit` is equivalent to:~~
~~`!conjunction_v<is_convertible<const UTypes&, Types>...>`~~

~~`template<class... UTypes> constexpr explicit(see below) tuple(tuple<UTypes...>&& u);`~~

21 ~~*Constraints:*~~

(21.1) — ~~`sizeof...(Types)` equals `sizeof...(UTypes)`, and~~

(21.2) — ~~`is_constructible_v<Ti, Ui>` is `true` for all *i*, and~~

(21.3) — ~~either `sizeof...(Types)` is not 1, or (when `Types...` expands to `T` and `UTypes...` expands to `U`) `is_convertible_v<tuple<U>, T>`, `is_constructible_v<T, tuple<U>>`, and `is_same_v<T, U>` are all `false`.~~

22 ~~*Effects:* For all *i*, initializes the $i^{th}$ element of `*this` with `std::forward<Ui>(get<i>(u))`.~~

23 ~~*Remarks:* The expression inside `explicit` is equivalent to:~~
~~`!conjunction_v<is_convertible<UTypes, Types>...>`~~

~~`template<class U1, class U2> constexpr explicit(see below) tuple(const pair<U1, U2>& u);`~~

24 ~~*Constraints:*~~

(24.1) — ~~`sizeof...(Types)` is 2,~~

(24.2) — ~~`is_constructible_v<T0, const U1&>` is `true`, and~~

(24.3) — ~~`is_constructible_v<T1, const U2&>` is `true`~~

25 ~~*Effects:* Initializes the first element with `u.first` and the second element with `u.second`.~~

26 ~~*Remarks:* The expression inside `explicit` is equivalent to: `!is_convertible_v<const U1&, T0> || !is_convertible_v<const U2&, T1>`~~

~~`template<class U1, class U2> constexpr explicit(`*see below*`) tuple(pair<U1, U2>&& u);`~~

27 ~~*Constraints:*~~

(27.1) — ~~`sizeof...(Types)` is 2,~~

(27.2) — ~~`is_constructible_v<T0, U1>` is `true`, and~~

(27.3) — ~~`is_constructible_v<T1, U2>` is `true`~~

28 ~~*Effects:* Initializes the first element with `std::forward<U1>(u.first)` and the second element with `std::forward<U2>(u.second)`.~~

29 ~~*Remarks:* The expression inside `explicit` is equivalent to: `!is_convertible_v<U1, T0> || !is_convertible_v<U2, T1>`~~

```
  template<class Alloc>
    constexpr explicit(see below)
      tuple(allocator_arg_t, const Alloc& a);
  template<class Alloc>
    constexpr explicit(see below)
      tuple(allocator_arg_t, const Alloc& a, const Types&...);
  template<class Alloc, class... UTypes>
    constexpr explicit(see below)
      tuple(allocator_arg_t, const Alloc& a, UTypes&&...);
  template<class Alloc>
    constexpr tuple(allocator_arg_t, const Alloc& a, const
        tuple&);
  template<class Alloc>
    constexpr tuple(allocator_arg_t, const Alloc& a, tuple&&);
+ template<class Alloc, class... UTypes>
+   constexpr explicit(see below)
+     tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&);
  template<class Alloc, class... UTypes>
    constexpr explicit(see below)
      tuple(allocator_arg_t, const Alloc& a, const
        tuple<UTypes...>&);
  template<class Alloc, class... UTypes>
```

```
    constexpr explicit(see below)
      tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&&);
+ template<class Alloc, class... UTypes>
+   constexpr explicit(see below)
+     tuple(allocator_arg_t, const Alloc& a, const
        tuple<UTypes...>&&);
+ template<class Alloc, class U1, class U2>
+   constexpr explicit(see below)
+     tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&);
  template<class Alloc, class U1, class U2>
    constexpr explicit(see below)
      tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&);
  template<class Alloc, class U1, class U2>
    constexpr explicit(see below)
      tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);
+ template<class Alloc, class U1, class U2>
+   constexpr explicit(see below)
+     tuple(allocator_arg_t, const Alloc& a, const pair<U1,
        U2>&&);
```

31 *Preconditions:* `Alloc` meets the *Cpp17Allocator* requirements (Table 38).

32 *Effects*: Equivalent to the preceding constructors except that each element is constructed with uses-allocator construction.

4. Add the following to 20.5.3.2 [tuple.assign]:

```
constexpr const tuple& operator=(const tuple& u) const;
```

? *Constraints*: `(is_copy_assignable_v<const Types> && ...)` is `true`.

? *Effects*: Assigns each element of `u` to the corresponding element of `*this`.

? *Returns*: `*this`.

```
constexpr const tuple& operator=(tuple&& u) const;
```

? *Constraints*: `(is_assignable_v<const Types&, Types> && ...)` is `true`.

? *Effects*: For all *i*, assigns `std::forward<T`$_i$`>(get<`*i*`>(u))` to `get<`*i*`>(*this)`.

? *Returns*: `*this`.

```
template<class... UTypes> constexpr const tuple& operator=(const
        tuple<UTypes...>& u) const;
```

? Constraints:

(?.1) — `sizeof...(Types)` equals `sizeof...(UTypes)` and

(?.2) — `(is_assignable_v<const Types&, const UTypes&> && ...)` is `true`.

? *Effects*: Assigns each element of `u` to the corresponding element of `*this`.

? *Returns*: `*this`.

```
template<class... UTypes> constexpr const tuple& operator=
        (tuple<UTypes...>&& u) const;
```

? Constraints:

(?.1) — `sizeof...(Types)` equals `sizeof...(UTypes)` and

(?.2) — `(is_assignable_v<const Types&, UTypes> && ...)` is `true`.

? *Effects*: For all *i*, assigns `std::forward<`$U_i$`>(get<`*i*`>(u))` to `get<`*i*`>(*this)`.

? *Returns*: `*this`.

```
template<class U1, class U2> constexpr const tuple& operator=(const pair<U1,
        U2>& u) const;
```

? *Constraints:*

(?.1) — `sizeof...(Types)` is 2,

(?.2) — `is_assignable_v<const` $T_0$`&, const U1&>` is `true`, and

(?.3) — `is_assignable_v<const` $T_1$`&, const U2&>` is `true`

? *Effects:* Assigns `u.first` to the first element and `u.second` to the second element.

? *Returns*: `*this`.

```
template<class U1, class U2> constexpr const tuple& operator=(pair<U1, U2>&&
        u) const;
```

? *Constraints:*

(?.1) — `sizeof...(Types)` is 2,

(?.2) — `is_assignable_v<const` $T_0$`&, U1>` is `true`, and

(?.3) — `is_assignable_v<const` $T_1$`&, U2>` is `true`

? *Effects:* Assigns `std::forward<U1>(u.first)` to the first element and `std::forward<U2>(u.second)` to the second element.

? *Returns*: `*this`.

5. Edit 20.5.3.3 [tuple.swap] as indicated:

```
      constexpr void swap(tuple& rhs) noexcept(see below);
    + constexpr void swap(const tuple& rhs) const noexcept(see below);
```

1 *Preconditions*: Each element in `*this` is swappable with (16.4.4.3 [swappable.requirements]) the corresponding element in `rhs`.

2 *Effects*: Calls `swap` for each element in `*this` and its corresponding element in `rhs`.

3 *Throws*: Nothing unless one of the element-wise `swap` calls throws an exception.

4 *Remarks*: The expression inside `noexcept` is equivalent to the logical AND of the following expressions: `is_nothrow_swappable_v<T_i>`, where `T_i` is the $i^{\text{th}}$ type in `Types`.

(4.1) — `(is_nothrow_swappable_v<Types> && ...)` for the first overload.

(4.2) — `(is_nothrow_swappable_v<const Types> && ...)` for the second overload.

6. Edit 20.5.10 [tuple.special] as indicated:

```
  template<class... Types>
    constexpr void swap(tuple<Types...>& x, tuple<Types...>& y)
        noexcept(see below);
+ template<class... Types>
+   constexpr void swap(const tuple<Types...>& x, const
        tuple<Types...>& y) noexcept(see below);
```

1 *Constraints*: `is_swappable_v<T>` is true for every type `T` in `Types`.

(1.1) — For the first overload, `(is_swappable_v<Types> && ...)` is `true`.

(1.2) — For the second overload, `(is_swappable_v<const Types> && ...)` is `true`.

2 *Effects*: As if by `x.swap(y)`.

3 *Remarks*: The expression inside `noexcept` is equivalent to: `noexcept(x.swap(y))`.

## § 5.2 `pair`

1. Edit 20.2.1 [utility.syn], header `<utility>` synopsis, as indicated:

```
#include <compare>              // see [compare.syn]
#include <initializer_list>     // see [initializer.list.syn]

namespace std {
  // [...]

  // [pairs], class template pair
  template<class T1, class T2>
    struct pair;

+  template<class T1, class T2, class U1, class U2, template<class> class
       TQual, template<class> class UQual>
```

```
+    requires requires { typename pair<common_reference_t<TQual<T1>,
        UQual<U1>>,
+                                      common_reference_t<TQual<T2>,
        UQual<U2>>>; }
+  struct basic_common_reference<pair<T1, T2>, pair<U1, U2>, TQual, UQual> {
+    using type = pair<common_reference_t<TQual<T1>, UQual<U1>>,
+                      common_reference_t<TQual<T2>, UQual<U2>>>;
+  };
+
+  template<class T1, class T2, class U1, class U2>
+    requires requires { typename pair<common_type_t<T1, U1>,
        common_type_t<T2, U2>>; }
+  struct common_type<pair<T1, T2>, pair<U1, U2>> {
+    using type = pair<common_type_t<T1, U1>, common_type_t<T2, U2>>;
+  };

   // [pairs.spec], pair specialized algorithms
   template<class T1, class T2>
     constexpr bool operator==(const pair<T1, T2>&, const pair<T1, T2>&);
   template<class T1, class T2>
     constexpr common_comparison_category_t<synth-three-way-result<T1>,
                                            synth-three-way-result<T2>>
       operator<=>(const pair<T1, T2>&, const pair<T1, T2>&);

   template<class T1, class T2>
     constexpr void swap(pair<T1, T2>& x, pair<T1, T2>& y)
       noexcept(noexcept(x.swap(y)));
+  template<class... Types>
+    constexpr void swap(const pair<T1, T2>& x, const pair<T1, T2>& y)
       noexcept(noexcept(x.swap(y)));
 }
```

2. Edit 20.4.2 [pairs.pair] as indicated:

```
namespace std {
  template<class T1, class T2>
  struct pair {
    using first_type  = T1;
    using second_type = T2;

    T1 first;
    T2 second;

    pair(const pair&) = default;
    pair(pair&&) = default;
    constexpr explicit(see below) pair();
    constexpr explicit(see below) pair(const T1& x, const T2& y);
    template<class U1, class U2>
      constexpr explicit(see below) pair(U1&& x, U2&& y);
+    template<class U1, class U2>
+      constexpr explicit(see below) pair(pair<U1, U2>& p);
    template<class U1, class U2>
      constexpr explicit(see below) pair(const pair<U1, U2>& p);
    template<class U1, class U2>
      constexpr explicit(see below) pair(pair<U1, U2>&& p);
+    template<class U1, class U2>
```

```
+      constexpr explicit(see below) pair(const pair<U1, U2>&& p);
     template<class... Args1, class... Args2>
       constexpr pair(piecewise_construct_t,
                      tuple<Args1...> first_args, tuple<Args2...>
         second_args);

     constexpr pair& operator=(const pair& p);
+    constexpr const pair& operator=(const pair& p) const;
     template<class U1, class U2>
       constexpr pair& operator=(const pair<U1, U2>& p);
+    template<class U1, class U2>
+      constexpr const pair& operator=(const pair<U1, U2>& p) const;
     constexpr pair& operator=(pair&& p) noexcept(see below);
+    constexpr const pair& operator=(pair&& p) const;
     template<class U1, class U2>
       constexpr pair& operator=(pair<U1, U2>&& p);
+    template<class U1, class U2>
+      constexpr const pair& operator=(pair<U1, U2>&& p) const;

     constexpr void swap(pair& p) noexcept(see below);
+    constexpr void swap(const pair& p) noexcept(see below);
   };

   template<class T1, class T2>
     pair(T1, T2) -> pair<T1, T2>;
 }
```

[…]

```
template<class U1, class U2> constexpr explicit(see below) pair(pair<U1,
        U2>& p);
template<class U1, class U2> constexpr explicit(see below) pair(const
        pair<U1, U2>& p);
template<class U1, class U2> constexpr explicit(see below) pair(pair<U1,
        U2>&& p);
template<class U1, class U2> constexpr explicit(see below) pair(const
        pair<U1, U2>&& p);
```

? Let `FWD(u)` be `static_cast<decltype(u)>(u)`.

? *Constraints*:

(17.1) — `is_constructible_v<first_type, decltype(get<0>(FWD(p)))>` is `true` and

(17.2) — `is_constructible_v<second_type, decltype(get<1>(FWD(p)))>` is `true`.

? *Effects:* Initializes `first` with `get<0>(FWD(p))` and `second` with `get<1>(FWD(p))`.

? *Remarks:* The expression inside `explicit` is equivalent to:
`!is_convertible_v<decltype(get<0>(FWD(p))), first_type> || !is_convertible_v<decltype(get<1>(FWD(p))), second_type>`.

~~template<class U1, class U2> constexpr explicit(*see below*) pair(const pair<U1, U2>& p);~~

14 ~~*Constraints:*~~

(14.1) — ~~is_constructible_v<first_type, const U1&> is true and~~

(14.2) — ~~is_constructible_v<second_type, const U2&> is true.~~

15 ~~*Effects:* Initializes members from the corresponding members of the argument.~~

16 ~~*Remarks:* The expression inside explicit is equivalent to:~~
~~!is_convertible_v<const U1&, first_type> || !is_convertible_v<const U2&, second_type>~~

~~template<class U1, class U2> constexpr explicit(*see below*) pair(pair<U1, U2>&& p);~~

17 ~~*Constraints:*~~

(17.1) — ~~is_constructible_v<first_type, U1> is true and~~

(17.2) — ~~is_constructible_v<second_type, U2> is true.~~

18 ~~*Effects:* Initializes first with std::forward<U1>(p.first) and second with std::forward<U2>(p.second).~~

19 ~~*Remarks:* The expression inside explicit is equivalent to:~~
~~!is_convertible_v<U1, first_type> || !is_convertible_v<U2, second_type>~~

```
constexpr const pair& operator=(const pair& p) const;
```

? *Constraints*:

(?.1) — `is_copy_assignable<const first_type>` is `true` and

(?.2) — `is_copy_assignable<const second_type>` is `true`.

? *Effects*: Assigns `p.first` to `first` and `p.second` to `second`.

? *Returns*: `*this`.

```
template<class U1, class U2> constexpr const pair& operator=(const pair<U1, U2>& p) const;
```

? *Constraints:*

(?.1) — `is_assignable_v<const first_type&, const U1&>` is `true`, and

(?.2) — `is_assignable_v<const second_type&, const U2&>` is `true`

? *Effects:* Assigns `p.first` to `first` and `p.second` to `second`.

? *Returns*: `*this`.

```
constexpr const pair& operator=(pair&& p) const;
```

? *Constraints*:

(?.1) — `is_assignable<const first_type&, first_type>` is `true` and

(?.2) — `is_assignable<const second_type&, second_type>` is `true`.

? *Effects*: Assigns `std::forward<first_type>(p.first)` to `first` and `std::forward<second_type>(p.second)` to `second`.

? *Returns*: `*this`.

```
template<class U1, class U2> constexpr const pair& operator=(pair<U1, U2>&&
        p) const;
```

? *Constraints:*

(?.1) — `is_assignable_v<const first_type&, U1>` is `true`, and

(?.2) — `is_assignable_v<const second_type&, U2>` is `true`

? *Effects:* Assigns `std::forward<U1>(p.first)` to `first` and `std::forward<U2>(u.second)` to `second`.

? *Returns*: `*this`.

```
  constexpr void swap(pair& p) noexcept(see below);
+ constexpr void swap(const pair& p) const noexcept(see below);
```

35 *Preconditions*: `first` is swappable with (16.4.4.3 [swappable.requirements]) `p.first` and `second` is swappable with `p.second`.

36 *Effects*: Swaps `first` with `p.first` and `second` with `p.second`.

37 *Remarks*: The expression inside `noexcept` is equivalent to

(4.1) — `is_nothrow_swappable_v<first_type> && is_nothrow_swappable_v<second_type>` for the first overload.

(4.2) — `is_nothrow_swappable_v<const first_type> && is_nothrow_swappable_v<const second_type>` for the second overload.

3. Edit 20.4.3 [pairs.spec] as indicated:

```
  template<class T1, class T2>
    constexpr void swap(pair<T1, T2>& x, pair<T1, T2>& y)
        noexcept(noexcept(x.swap(y)));
+ template<class T1, class T2>
+   constexpr void swap(const pair<T1, T2>& x, const pair<T1, T2>&
        y) noexcept(noexcept(x.swap(y)));
```

3 *Constraints*:

(3.1) — For the first overload, `is_swappable_v<T1>` is `true` and `is_swappable_v<T2>` is `true`.

(3.2) — For the second overload, `is_swappable_v<const T1>` is `true` and `is_swappable_v<const T2>` is `true`.

3 *Effects*: Equivalent to `x.swap(y)`.

4. Edit 20.10.2 [memory.syn], header `<memory>` synopsis, as indicated:

```
#include <compare>              // see [compare.syn]

namespace std {
  // [...]

  // [allocator.uses.construction], uses-allocator construction
  template<class T, class Alloc, class... Args>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    Args&&... args)
        noexcept -> see below;
  template<class T, class Alloc, class Tuple1, class Tuple2>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
        piecewise_construct_t,
                                                    Tuple1&& x, Tuple2&& y)
                                                    noexcept -> see below;
  template<class T, class Alloc>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc)
        noexcept -> see below;
  template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    U&& u, V&& v) noexcept
        -> see below;
+ template<class T, class Alloc, class U, class V>
+   constexpr auto uses_allocator_construction_args(const Alloc& alloc,
+                                                   pair<U,V>& pr) noexcept
        -> see below;
  template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    const pair<U,V>& pr)
        noexcept -> see below;
  template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    pair<U,V>&& pr)
        noexcept -> see below;
+ template<class T, class Alloc, class U, class V>
+   constexpr auto uses_allocator_construction_args(const Alloc& alloc,
+                                                   const pair<U,V>&& pr)
        noexcept -> see below;
  template<class T, class Alloc, class... Args>
    constexpr T make_obj_using_allocator(const Alloc& alloc, Args&&...
        args);
  template<class T, class Alloc, class... Args>
    constexpr T* uninitialized_construct_using_allocator(T* p, const Alloc&
        alloc,
                                                         Args&&... args);
```

```
  // [...]
}
```

5. Edit 20.10.8.2 [allocator.uses.construction] as indicated:

```
+ template<class T, class Alloc, class U, class V>
+   constexpr auto uses_allocator_construction_args(const Alloc&
        alloc,
+                                                   pair<U,V>& pr)
        noexcept -> see below;
  template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc&
        alloc,
                                                    const
        pair<U,V>& pr) noexcept -> see below;
```

12 *Constraints*: `T` is a specialization of `pair`.

13 *Effects*: Equivalent to:

```
     return uses_allocator_construction_args<T>(alloc,
             piecewise_construct,

             forward_as_tuple(pr.first),

             forward_as_tuple(pr.second));

  template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc&
        alloc,
                                                    pair<U,V>&&
        pr) noexcept -> see below;
+ template<class T, class Alloc, class U, class V>
+   constexpr auto uses_allocator_construction_args(const Alloc&
        alloc,
+                                                   const
        pair<U,V>&& pr) noexcept -> see below;
```

14 *Constraints*: `T` is a specialization of `pair`.

15 *Effects*: Equivalent to:

```
     return uses_allocator_construction_args<T>(alloc,
             piecewise_construct,

             forward_as_tuple(std::move(pr).firstget<0>
             (std::move(pr))),

             forward_as_tuple(std::move(pr).secondget<1>
             (std::move(pr))));
```

## § 5.3 `vector<bool>::reference`

Edit 22.3.12 [vector.bool], class template partial specialization `vector<bool, Allocator>` synopsis, as indicated:

```
namespace std {
  template<class Allocator>
  class vector<bool, Allocator> {
  public:
    // [...]

    // bit reference
    class reference {
      friend class vector;
      constexpr reference() noexcept;
    public:
      constexpr reference(const reference&) = default;
      constexpr ~reference();
      constexpr operator bool() const noexcept;
      constexpr reference& operator=(const bool x) noexcept;
      constexpr reference& operator=(const reference& x) noexcept;
+     constexpr const reference& operator=(bool x) const noexcept;
      constexpr void flip() noexcept;   // flips the bit
    };

    // [...]
  };
}
```

## § 5.4 Addition to <ranges>

Add the following to 24.2 [ranges.syn], header <ranges> synopsis:

```
// [...]
namespace std::ranges {
  // [...]

  // [range.zip], zip view
  template<input_range... Views>
      requires (view<Views> && ...) && (sizeof...(Views) > 0)
  class zip_view;

  template<class... Views>
  inline constexpr bool enable_borrowed_range<zip_view<Views...>>
      = (enable_borrowed_range<Views> && ...);

  namespace views { inline constexpr unspecified zip = unspecified; }

  // [range.zip.transform], zip transform view
  template<copy_constructible F, input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0) && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<Views>...> &&
             can-reference<invoke_result_t<F&, range_reference_t<Views>...>>
  class zip_transform_view;

  namespace views { inline constexpr unspecified zip_transform = unspecified; }

  // [range.adjacent], adjacent view
  template<forward_range V, size_t N>
      requires view<V> && (N > 0)
```

```
  class adjacent_view;

  template<class V, size_t N>
  inline constexpr bool enable_borrowed_range<adjacent_view<V, N>>
      = enable_borrowed_range<V>;

  namespace views {
    template<size_t N>
    inline constexpr unspecified adjacent = unspecified;
    inline constexpr auto pairwise = adjacent<2>;
  }

  // [range.adjacent.transform], adjacent transform view
  template<forward_range V, copy_constructible F, size_t N>
    requires see below
  class adjacent_transform_view;

  namespace views {
    template<size_t N>
    inline constexpr unspecified adjacent_transform = unspecified;
    inline constexpr auto pairwise_transform = adjacent_transform<2>;
  }
}
```

## § 5.5 `zip`

Add the following subclause to 24.7 [range.adaptors].

### § 24.7.? Zip view [range.zip]

#### § 24.7.?.1 Overview [range.zip.overview]

1 `zip_view` takes any number of `view`s and produces a `view` that iterates over these views in parallel, in the form of a `tuple` or `pair`.

2 The name `views::zip` denotes a customization point object (16.3.3.3.6 [customization.point.object]). Given a pack of subexpressions `Es...`, the expression `views::zip(Es...)` is expression-equivalent to

.1) — *decay-copy*`(views::empty<tuple<>>)` if `Es` is an empty pack,

.2) — otherwise, `zip_view<views::all_t<decltype((Es))>...>(Es...)`.

#### § 24.7.?.2 Class template `zip_view` [range.zip.view]

```
namespace std::ranges {

template<class... Rs>
concept zip-is-common =              // exposition only
  (sizeof...(Rs) == 1 && (common_range<Rs> && ...)) ||
  (!(bidirectional_range<Rs> && ...) && (common_range<Rs> && ...)) ||
  ((random_access_range<Rs> && ...) && (sized_range<Rs> && ...));

template<class... Ts>
```

```
using tuple-or-pair = see below;  // exposition only

template<class F, class Tuple>
constexpr auto tuple-transform(F&& f, Tuple&& tuple) // exposition only
{
  return apply([&]<class... Ts>(Ts&&... elements){
    return tuple-or-pair<invoke_result_t<F&, Ts>...>(
      invoke(f, std::forward<Ts>(elements))...
    );
  }, std::forward<Tuple>(tuple));
}

template<class F, class Tuple>
constexpr void tuple-for-each(F&& f, Tuple&& tuple) // exposition only
{
  apply([&]<class... Ts>(Ts&&... elements){
    (invoke(f, std::forward<Ts>(elements)), ...);
  }, std::forward<Tuple>(tuple));
}

template<input_range... Views>
  requires (view<Views> && ...) && (sizeof...(Views) > 0)
class zip_view : public view_interface<zip_view<Views...>>{
  tuple<Views...> views_;                // exposition only

  template<bool> class iterator;         // exposition only
  template<bool> class sentinel;         // exposition only

public:
  constexpr zip_view() = default;
  constexpr explicit zip_view(Views... views);

  constexpr auto begin() requires (!(simple-view<Views> && ...)){
    return iterator<false>(tuple-transform(ranges::begin, views_));
  }
  constexpr auto begin() const requires (range<const Views> && ...){
    return iterator<true>(tuple-transform(ranges::begin, views_));
  }

  constexpr auto end() requires (!(simple-view<Views> && ...) && !zip-is-
        common<Views...>){
    return sentinel<false>(tuple-transform(ranges::end, views_));
  }
  constexpr auto end() requires (!(simple-view<Views> && ...) && zip-is-
        common<Views...>){
    if constexpr ((random_access_range<Views> && ...)){
      return begin() + size();
    }
    else {
      return iterator<false>(tuple-transform(ranges::end, views_));
    }
  }

  constexpr auto end() const requires ((range<const Views> && ...) && !zip-is-
        common<const Views...>){
    return sentinel<true>(tuple-transform(ranges::end, views_));
```

```
  }
  constexpr auto end() const requires ((range<const Views> && ...) && zip-is-
        common<const Views...>){
    if constexpr ((random_access_range<const Views> && ...)){
      return begin() + size();
    }
    else {
      return iterator<true>(tuple-transform(ranges::end, views_));
    }
  }

  constexpr auto size() requires (sized_range<Views> && ...);
  constexpr auto size() const requires (sized_range<const Views> && ...);
};

template<class... Rs>
  zip_view(Rs&&...) -> zip_view<views::all_t<Rs>...>;

}
```

1 Given some pack of types `Ts`, the alias template *`tuple-or-pair`* is defined as follows:

.1) — If `sizeof...(Ts)` is 2, *`tuple-or-pair`*`<Ts...>` denotes `pair<Ts...>`.

.2) — Otherwise, *`tuple-or-pair`*`<Ts...>` denotes `tuple<Ts...>`.

2 Two `zip_view` objects have the same underlying sequence only if and only if the corresponding elements of *`views_`* are equal (18.2 [concepts.equality]) and have the same underlying sequence. [ *Note ?:* In particular, comparison of iterators obtained from `zip_view` objects that do not have the same underlying sequence is not required to produce meaningful results (23.3.4.11 [iterator.concept.forward]). — *end note* ]

```
constexpr explicit zip_view(Views... views);
```

3 *Effects*: Initializes *`views_`* with `std::move(views)...`.

```
constexpr auto size() requires (sized_range<Views> && ...);
constexpr auto size() const requires (sized_range<const Views> && ...);
```

4 *Effects*: Equivalent to:

```
return apply([](auto... sizes){
  return ranges::min({
    common_type_t<decltype(sizes)...>(sizes)...
  });
}, tuple-transform(ranges::size, views_));
```

§ **24.7.?.3 Class template `zip_view::`*`iterator`* [range.zip.iterator]**

```
namespace std::ranges {
  template<input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0)
  template<bool Const>
  class zip_view<Views...>::iterator {
```

```
  tuple-or-pair<iterator_t<maybe-const<Const, Views>>...> current_;
      // exposition only
  constexpr explicit iterator(tuple-or-pair<iterator_t<maybe-const<Const,
      Views>>...> current); // exposition only
public:
  using iterator_category = input_iterator_tag; // not always present
  using iterator_concept  = see below;
  using value_type = tuple-or-pair<range_value_t<maybe-const<Const, Views>>...>;
  using difference_type = common_type_t<range_difference_t<maybe-const<Const,
      Views>>...>;

  iterator() = default;
  constexpr iterator(iterator<!Const> i)
    requires Const && (convertible_to<iterator_t<Views>, iterator_t<maybe-
      const<Const, Views>>> && ...);

  constexpr auto operator*() const;
  constexpr iterator& operator++();
  constexpr void operator++(int);
  constexpr iterator operator++(int) requires (forward_range<maybe-const<Const,
      Views>> && ...);

  constexpr iterator& operator--() requires (bidirectional_range<maybe-const<Const,
      Views>> && ...);
  constexpr iterator operator--(int) requires (bidirectional_range<maybe-
      const<Const, Views>> && ...);

  constexpr iterator& operator+=(difference_type x)
    requires (random_access_range<maybe-const<Const, Views>> && ...);
  constexpr iterator& operator-=(difference_type x)
    requires (random_access_range<maybe-const<Const, Views>> && ...);

  constexpr auto operator[](difference_type n) const
    requires (random_access_range<maybe-const<Const, Views>> && ...);

  friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires (equality_comparable<iterator_t<maybe-const<Const, Views>>> && ...);

  friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, Views>> && ...);
  friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, Views>> && ...);
  friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, Views>> && ...);
  friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, Views>> && ...);
  friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, Views>> && ...) &&
             (three_way_comparable<iterator_t<maybe-const<Const, Views>>> && ...);

  friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires (random_access_range<maybe-const<Const, Views>> && ...);
  friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires (random_access_range<maybe-const<Const, Views>> && ...);
  friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires (random_access_range<maybe-const<Const, Views>> && ...);
```

```
    friend constexpr difference_type operator-(const iterator& x, const iterator& y)
      requires (sized_sentinel_for<iterator_t<maybe-const<Const, Views>>,
        iterator_t<maybe-const<Const, Views>>> && ...);

    friend constexpr auto iter_move(const iterator& i)
      noexcept(see below);

    friend constexpr void iter_swap(const iterator& l, const iterator& r)
      noexcept(see below)
      requires (indirectly_swappable<iterator_t<maybe-const<Const, Views>>> && ...);
  };
}
```

1 *iterator*::iterator_concept is defined as follows:

.1) — If every type in Views models random_access_range, then iterator_concept denotes random_access_iterator_tag.

.2) — Otherwise, if every type in Views models bidirectional_range, then iterator_concept denotes bidirectional_iterator_tag.

.3) — Otherwise, if every type in Views models forward_range, then iterator_concept denotes forward_iterator_tag.

.4) — Otherwise, iterator_concept denotes input_iterator_tag.

2 *iterator*::iterator_category is present if and only if every type in Views models forward_range.

```
constexpr explicit iterator(tuple-or-pair<iterator_t<maybe-const<Const, Views>>...>
        current);
```

3 *Effects*: Initializes *current_* with std::move(current).

```
constexpr iterator(iterator<!Const> i)
  requires Const && (convertible_to<iterator_t<Views>, iterator_t<maybe-const<Const,
        Views>>> && ...);
```

4 *Effects*: Initializes *current_* with std::move(i.*current_*).

```
  constexpr auto operator*() const;
```

5 *Effects*: Equivalent to:

```
        return tuple-transform([](auto& i) -> decltype(auto) { return
              *i; }, current_);
```

```
constexpr iterator& operator++();
```

6 *Effects*: Equivalent to:

```
        tuple-for-each([](auto& i) { ++i; }, current_);
        return *this;
```

```
constexpr void operator++(int);
```

7 *Effects*: Equivalent to ++*this;.

```
constexpr iterator operator++(int) requires (forward_range<maybe-const<Const, Views>>
        && ...);
```

8 *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
constexpr iterator& operator--() requires (bidirectional_range<maybe-const<Const,
        Views>> && ...);
```

9 *Effects*: Equivalent to:

```
tuple-for-each([](auto& i) { --i; }, current_);
return *this;
```

```
constexpr iterator operator--(int) requires (bidirectional_range<maybe-const<Const,
        Views>> && ...);
```

10 *Effects*: Equivalent to:

```
auto tmp = *this;
--*this;
return tmp;
```

```
constexpr iterator& operator+=(difference_type x)
  requires (random_access_range<maybe-const<Const, Views>> && ...);
```

11 *Effects*: Equivalent to:

```
tuple-for-each([](auto& i) { i += x; }, current_);
return *this;
```

```
  constexpr iterator& operator-=(difference_type x)
    requires (random_access_range<maybe-const<Const, Views>> && ...);
```

12 *Effects*: Equivalent to:

```
tuple-for-each([](auto& i) { i -= x; }, current_);
return *this;
```

```
constexpr auto operator[](difference_type n) const
  requires (random_access_range<maybe-const<Const, Views>> && ...);
```

13 *Effects*: Equivalent to:

```
return tuple-transform([&](auto& i) -> decltype(auto) { return
      i[n]; }, current_);
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
  requires (equality_comparable<iterator_t<maybe-const<Const, Views>>> && ...);
```

14 *Returns*:

(14.1) — `x.current_ == y.current_` if `(bidirectional_range<maybe-const<Const, Views>> && ...)` is `true`.

(14.2) — Otherwise, `true` if there exists an integer $0 \le i < $ `sizeof...(Views)` such that `bool(get<i>(x.current_) == get<i>(y.current_))` is `true`.

(14.3) — Otherwise, `false`.

15 [ *Note ?:* For non-bidirectional views, the use of logical OR on the result of individual comparisons allows `zip_view` to model `common_range` when all constituent views model `common_range`. — *end note* ]

```
friend constexpr bool operator<(const iterator& x, const iterator& y)
  requires (random_access_range<maybe-const<Const, Views>> && ...);
```

16 *Effects*: Equivalent to: `return x.current_ < y.current_;`

```
friend constexpr bool operator>(const iterator& x, const iterator& y)
  requires (random_access_range<maybe-const<Const, Views>> && ...);
```

17 *Effects*: Equivalent to: `return y < x;`

```
friend constexpr bool operator<=(const iterator& x, const iterator& y)
  requires (random_access_range<maybe-const<Const, Views>> && ...);
```

18 *Effects*: Equivalent to: `return !(y < x);`

```
friend constexpr bool operator>=(const iterator& x, const iterator& y)
  requires (random_access_range<maybe-const<Const, Views>> && ...);
```

19 *Effects*: Equivalent to: `return !(x < y);`

```
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
  requires (random_access_range<maybe-const<Const, Views>> && ...) &&
           (three_way_comparable<iterator_t<maybe-const<Const, Views>>> && ...);
```

20 *Effects*: Equivalent to: `return x.current_ <=> y.current_;`

```
friend constexpr iterator operator+(const iterator& i, difference_type n)
  requires (random_access_range<maybe-const<Const, Views>> && ...);
friend constexpr iterator operator+(difference_type n, const iterator& i)
  requires (random_access_range<maybe-const<Const, Views>> && ...);
```

21 *Effects*: Equivalent to:

```
return iterator(tuple-transform([&](auto& it) { return it + n;
      }, i.current_));
```

```
friend constexpr iterator operator-(const iterator& i, difference_type n)
  requires (random_access_range<maybe-const<Const, Views>> && ...);
```

22 *Effects*: Equivalent to:

```
return iterator(tuple-transform([&](auto& it) { return it - n;
      }, i.current_));
```

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
  requires (sized_sentinel_for<iterator_t<maybe-const<Const, Views>>,
        iterator_t<maybe-const<Const, Views>>> && ...);
```

23 Let *DIST(i)* be `get<i>(x.current_) - get<i>(y.current_)`.

24 *Returns:* The value with the smallest absolute value among *DIST(n)* for all integers $0 \le n <$ `sizeof...(Views)`,

```
friend constexpr auto iter_move(const iterator& i)
  noexcept(see below);
```

25 *Effects*: Equivalent to:

```
return tuple-transform(ranges::iter_move, i.current_);
```

26 *Remarks:* The expression within `noexcept` is equivalent to

```
(noexcept(ranges::iter_move(declval<iterator_t<maybe-
      const<Const, Views>>>())) && ...) &&
(is_nothrow_move_constructible_v<range_rvalue_reference_t<maybe-
      const<Const, Views>>> && ...)
```

```
friend constexpr void iter_swap(const iterator& l, const iterator& r)
  noexcept(see below)
  requires (indirectly_swappable<iterator_t<maybe-const<Const, Views>>> && ...);
```

27 *Effects*: For every integer *i* in `[0, sizeof...(Views))`, performs `ranges::iter_swap(get<i>(l.current_), get<i>(r.current_))`.

28 *Remarks:* The expression within `noexcept` is equivalent to the logical AND of the following expressions:

```
noexcept(ranges::iter_swap(get<i>(l.current_), get<i>
      (r.current_)))
```

for every integer *i* in `[0, sizeof...(Views))`.

### § 24.7.?.4 Class template `zip_view::sentinel` [range.zip.sentinel]

```
namespace std::ranges {
  template<input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0)
  template<bool Const>
  class zip_view<Views...>::sentinel {
    tuple-or-pair<sentinel_t<maybe-const<Const, Views>>...> end_;
        // exposition only
    constexpr explicit sentinel(tuple-or-pair<sentinel_t<maybe-const<Const,
        Views>>...> end);    // exposition only
  public:
    sentinel() = default;
    constexpr sentinel(sentinel<!Const> i)
      requires Const && (convertible_to<sentinel_t<Views>, sentinel_t<maybe-
        const<Const, Views>>> && ...);
```

```
    template<bool OtherConst>
      requires (sentinel_for<sentinel_t<maybe-const<Const, Views>>, iterator_t<maybe-
        const<OtherConst, Views>>> && ...)
    friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel&
        y);

    template<bool OtherConst>
      requires (sized_sentinel_for<sentinel_t<maybe-const<Const, Views>>,
        iterator_t<maybe-const<OtherConst, Views>>> && ...)
    friend constexpr common_type_t<range_difference_t<maybe-const<OtherConst,
        Views>>...>
      operator-(const iterator<OtherConst>& x, const sentinel& y);

    template<bool OtherConst>
      requires (sized_sentinel_for<sentinel_t<maybe-const<Const, Views>>,
        iterator_t<maybe-const<OtherConst, Views>>> && ...)
    friend constexpr common_type_t<range_difference_t<maybe-const<OtherConst,
        Views>>...>
      operator-(const sentinel& y, const iterator<OtherConst>& x);
  };
}
```

```
constexpr explicit sentinel(tuple-or-pair<sentinel_t<maybe-const<Const, Views>>...>
        end);
```

1 *Effects*: Initializes *end_* with `end`.

```
constexpr sentinel(sentinel<!Const> i)
  requires Const && (convertible_to<sentinel_t<Views>, sentinel_t<maybe-const<Const,
        Views>>> && ...);
```

2 *Effects*: Initializes *end_* with `std::move(i.end_)`.

```
template<bool OtherConst>
  requires (sentinel_for<sentinel_t<maybe-const<Const, Views>>, iterator_t<maybe-
        const<OtherConst, Views>>> && ...)
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
```

3 *Returns*: `true` if there exists an integer $0 \le i <$ `sizeof...(Views)` such that `bool(get<i>(x.current_) == get<i>(y.end_))` is `true`. Otherwise, `false`.

```
template<bool OtherConst>
  requires (sized_sentinel_for<sentinel_t<maybe-const<Const, Views>>,
        iterator_t<maybe-const<OtherConst, Views>>> && ...)
friend constexpr common_type_t<range_difference_t<maybe-const<OtherConst, Views>>...>
  operator-(const iterator<OtherConst>& x, const sentinel& y);
```

4 Let *DIST(i)* be `get<i>(x.current_) - get<i>(y.end_)`.

5 *Returns:* The value with the smallest absolute value among *DIST(n)* for all integers $0 \le n <$ `sizeof...(Views)`,

```
template<bool OtherConst>
  requires (sized_sentinel_for<sentinel_t<maybe-const<Const, Views>>,
        iterator_t<maybe-const<OtherConst, Views>>> && ...)
```

```
friend constexpr common_type_t<range_difference_t<maybe-const<OtherConst, Views>>...>
  operator-(const sentinel& y, const iterator<OtherConst>& x);
```

6 *Effects*: Equivalent to `return -(x - y);`

## § 5.6 `zip_transform`

Add the following subclause to 24.7 [range.adaptors].

### § 24.7.? Zip transform view [range.zip.transform]

#### § 24.7.?.1 Overview [range.zip.transform.overview]

1 `zip_transform_view` takes an invocable object and any number of `views` and produces a `view` whose $M^{th}$ element is the result of applying the invocable object to the $M^{th}$ elements of all views.

2 The name `views::zip_transform` denotes a customization point object (16.3.3.3.6 [customization.point.object]). Given a subexpression `F` and a pack of subexpressions `Es...`,

.1) — if `Es` is an empty pack, let `FD` be `decay_t<decltype((F))>`.

.1) — if `copy_constructible<FD> && regular_invocable<FD&>` is `false`, `views::zip_transform(F, Es...)` is ill-formed.

.2) — Otherwise, the expression `views::zip_transform(F, Es...)` is expression-equivalent to `(void)F, decay-copy(views::empty<decay_t<invoke_result_t<FD&>>>)`.

.2) — Otherwise, the expression `views::zip_transform(F, Es...)` is expression-equivalent to`zip_transform_view(F, Es...)`.

#### § 24.7.?.2 Class template `zip_transform_view` [range.zip.transform.view]

```
namespace std::ranges {

template<copy_constructible F, input_range... Views>
  requires (view<Views> && ...) && (sizeof...(Views) > 0) && is_object_v<F> &&
           regular_invocable<F&, range_reference_t<Views>...> &&
           can-reference<invoke_result_t<F&, range_reference_t<Views>...>>
class zip_transform_view : public view_interface<zip_transform_view<F, Views...>> {
  semiregular-box<F> fun_;              // exposition only
  zip_view<Views...> zip_;              // exposition only

  using InnerView = zip_view<Views...>;  // exposition only
  template<bool Const>
  using ziperator = iterator_t<maybe-const<Const, InnerView>>;  // exposition only
  template<bool Const>
  using zentinel = sentinel_t<maybe-const<Const, InnerView>>;   // exposition only

  template<bool> class iterator;        // exposition only
  template<bool> class sentinel;        // exposition only

public:
  constexpr zip_transform_view() = default;
```

```
  constexpr explicit zip_transform_view(F fun, Views... views);

  constexpr auto begin() {
    return iterator<false>(*this, zip_.begin());
  }

  constexpr auto begin() const
    requires range<const InnerView> &&
             regular_invocable<const F&, range_reference_t<const Views>...>
  {
    return iterator<true>(*this, zip_.begin());
  }

  constexpr auto end() {
    return sentinel<false>(zip_.end());
  }

  constexpr auto end() requires common_range<InnerView> {
    return iterator<false>(*this, zip_.end());
  }

  constexpr auto end() const
    requires range<const InnerView> &&
             regular_invocable<const F&, range_reference_t<const Views>...>
  {
    return sentinel<true>(zip_.end());
  }

  constexpr auto end() const
    requires common_range<const InnerView> &&
             regular_invocable<const F&, range_reference_t<const Views>...>
  {
    return iterator<true>(*this, zip_.end());
  }

  constexpr auto size() requires sized_range<InnerView> {
    return zip_.size();
  }

  constexpr auto size() const requires sized_range<const InnerView> {
    return zip_.size();
  }
};

template<class F, class... Rs>
  zip_transform_view(F, Rs&&...) -> zip_transform_view<F, views::all_t<Rs>...>;

}
```

```
constexpr explicit zip_transform_view(F fun, Views... views);
```

1 *Effects*: Initializes *fun_* with `std::move(fun)` and *zip_* with `std::move(views)...`.

§ **24.7.?.3 Class template `zip_transform_view::`*`iterator`* [range.zip.transform.iterator]**

```
namespace std::ranges {
  template<copy_constructible F, input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0) && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<Views>...> &&
             can-reference<invoke_result_t<F&, range_reference_t<Views>...>>
  template<bool Const>
  class zip_transform_view<F, Views...>::iterator {
    using Parent = maybe-const<Const, zip_transform_view>;      // exposition only
    using Base = maybe-const<Const, InnerView>;                 // exposition only
    Parent* parent_ = nullptr;                                  // exposition only
    ziperator<Const> inner_ = ziperator<Const>();               // exposition only

    constexpr iterator(Parent& parent, ziperator<Const> inner); // exposition only

  public:
    using iterator_category = see below;                        // not always present
    using iterator_concept  = typename ziperator<Const>::iterator_concept;
    using value_type =
      remove_cvref_t<invoke_result_t<maybe-const<Const, F>&, range_reference_t<maybe-
        const<Const, Views>>...>>;
    using difference_type = range_difference_t<Base>;

    iterator() = default;
    constexpr iterator(iterator<!Const> i)
      requires Const && convertible_to<ziperator<false>, ziperator<Const>>;

    constexpr decltype(auto) operator*() const noexcept(see below);
    constexpr iterator& operator++();
    constexpr void operator++(int);
    constexpr iterator operator++(int) requires forward_range<Base>;

    constexpr iterator& operator--() requires bidirectional_range<Base>;
    constexpr iterator operator--(int) requires bidirectional_range<Base>;

    constexpr iterator& operator+=(difference_type x) requires
        random_access_range<Base>;
    constexpr iterator& operator-=(difference_type x) requires
        random_access_range<Base>;

    constexpr decltype(auto) operator[](difference_type n) const requires
        random_access_range<Base>;

    friend constexpr bool operator==(const iterator& x, const iterator& y)
      requires equality_comparable<ziperator<Const>>;

    friend constexpr bool operator<(const iterator& x, const iterator& y)
      requires random_access_range<Base>;
    friend constexpr bool operator>(const iterator& x, const iterator& y)
      requires random_access_range<Base>;
    friend constexpr bool operator<=(const iterator& x, const iterator& y)
      requires random_access_range<Base>;
    friend constexpr bool operator>=(const iterator& x, const iterator& y)
      requires random_access_range<Base>;
    friend constexpr auto operator<=>(const iterator& x, const iterator& y)
      requires random_access_range<Base> && three_way_comparable<ziperator<Const>>;
```

```
    friend constexpr iterator operator+(const iterator& i, difference_type n)
      requires random_access_range<Base>;
    friend constexpr iterator operator+(difference_type n, const iterator& i)
      requires random_access_range<Base>;
    friend constexpr iterator operator-(const iterator& i, difference_type n)
      requires random_access_range<Base>;
    friend constexpr difference_type operator-(const iterator& x, const iterator& y)
      requires sized_sentinel_for<ziperator<Const>, ziperator<Const>>;
  };
}
```

1 The member *typedef-name* `iterator::iterator_category` is defined if and only if *`Base`* models `forward_range`. In that case, `iterator::iterator_category` is defined as follows:

— If `invoke_result_t<maybe-const<Const, F>&, range_reference_t<maybe-const<Const, Views>>...>` is not an lvalue reference, `iterator_category` denotes `input_iterator_tag`.

— Otherwise, let `Cs...` denote the pack of types `iterator_traits<iterator_t<maybe-const<Const, Views>>>::iterator_category...`.

  — If `(derived_from<Cs, random_access_iterator_tag> && ...)` is `true`, `iterator_category` denotes `random_access_iterator_tag`;

  — Otherwise, if `(derived_from<Cs, bidirectional_iterator_tag> && ...)` is `true`, `iterator_category` denotes `bidirectional_iterator_tag`;

  — Otherwise, if `(derived_from<Cs, forward_iterator_tag> && ...)` is `true`, `iterator_category` denotes `forward_iterator_tag`;

  — Otherwise, `iterator_category` denotes `input_iterator_tag`.

```
constexpr iterator(Parent& parent, ziperator<Const> inner);
```

2 *Effects*: Initializes *`parent_`* with `addressof(parent)` and *`inner_`* with `std::move(inner)`.

```
constexpr iterator(iterator<!Const> i)
  requires Const && convertible_to<ziperator<false>, ziperator<Const>>;
```

3 *Effects*: Initializes *`parent_`* with `i.`*`parent_`* and *`inner_`* with `std::move(i.`*`inner_`*`)`.

```
constexpr decltype(auto) operator*() const noexcept(see below);
```

4 *Effects*: Equivalent to:

```
return apply([&](const auto&... iters) -> decltype(auto) {
  return invoke(*parent_->fun_, *iters...);
}, inner_.current_);
```

5 *Remarks:* Let `Is` be the pack `0, 1, ..., (sizeof...(Views)-1)`. The expression within `noexcept` is equivalent to `noexcept(invoke(*`*`parent_->fun_`*`, *get<Is>(`*`inner_.current_`*`)...))`.

```
constexpr iterator& operator++();
```

6 *Effects*: Equivalent to:

```
        ++inner_;
        return *this;
```

```
constexpr void operator++(int);
```

7 *Effects*: Equivalent to `++*this;`.

```
constexpr iterator operator++(int) requires forward_range<Base>;
```

8 *Effects*: Equivalent to:

```
        auto tmp = *this;
        ++*this;
        return tmp;
```

```
constexpr iterator& operator--() requires bidirectional_range<Base>;
```

9 *Effects*: Equivalent to:

```
        --inner_;
        return *this;
```

```
constexpr iterator operator--(int) requires bidirectional_range<Base>;
```

10 *Effects*: Equivalent to:

```
        auto tmp = *this;
        --*this;
        return tmp;
```

```
constexpr iterator& operator+=(difference_type x)
  requires random_access_range<Base>;
```

11 *Effects*: Equivalent to:

```
        inner_ += x;
        return *this;
```

```
constexpr iterator& operator-=(difference_type x)
  requires random_access_range<Base>;
```

12 *Effects*: Equivalent to:

```
        inner_ -= x;
        return *this;
```

```
constexpr decltype(auto) operator[](difference_type n) const
  requires random_access_range<Base>;
```

13 *Effects*: Equivalent to:

```
        return apply([&](const auto&... iters) -> decltype(auto) {
          return invoke(*parent_->fun_, iters[n]...);
        }, inner_.current_);
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
  requires equality_comparable<ziperator<Const>>;
friend constexpr bool operator<(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
  requires random_access_range<Base> && three_way_comparable<ziperator<Const>>;
```

14 Let *op* be the operator.

15 *Effects*: Equivalent to: `return x.inner_ op y.inner_;`

```
friend constexpr iterator operator+(const iterator& i, difference_type n)
  requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
  requires random_access_range<Base>;
```

16 *Effects*: Equivalent to: `return iterator(*i.parent_, i.inner_ + n);`

```
friend constexpr iterator operator-(const iterator& i, difference_type n)
  requires random_access_range<Base>;
```

17 *Effects*: Equivalent to: `return iterator(*i.parent_, i.inner_ - n);`

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
  requires sized_sentinel_for<ziperator<Const>, ziperator<Const>>;
```

18 *Effects*: Equivalent to: `return x.inner_ - y.inner_;`

### § 24.7.?.4 Class template `zip_transform_view::sentinel` [range.zip.transform.sentinel]

```
namespace std::ranges {
  template<copy_constructible F, input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0) && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<Views>...> &&
             can-reference<invoke_result_t<F&, range_reference_t<Views>...>>
  template<bool Const>
  class zip_transform_view<F, Views...>::sentinel {
    zentinel<Const> inner_;                              // exposition only
    constexpr explicit sentinel(zentinel<Const> inner); // exposition only
  public:
    sentinel() = default;
    constexpr sentinel(sentinel<!Const> i)
      requires Const && convertible_to<zentinel<false>, zentinel<Const>>;

    template<bool OtherConst>
      requires sentinel_for<zentinel<Const>, ziperator<OtherConst>>
    friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel&
        y);
```

```
    template<bool OtherConst>
      requires sized_sentinel_for<zentinel<Const>, ziperator<OtherConst>>
    friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
      operator-(const iterator<OtherConst>& x, const sentinel& y);

    template<bool OtherConst>
      requires sized_sentinel_for<zentinel<Const>, ziperator<OtherConst>>
    friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
      operator-(const sentinel& x, const iterator<OtherConst>& y);
  };
}
```

```
constexpr explicit sentinel(zentinel<Const> inner);
```

1 *Effects*: Initializes *inner_* with `inner`.

```
constexpr sentinel(sentinel<!Const> i)
  requires Const && convertible_to<zentinel<false>, zentinel<Const>>;
```

2 *Effects*: Initializes *inner_* with `std::move(i.inner_)`.

```
template<bool OtherConst>
  requires sentinel_for<zentinel<Const>, ziperator<OtherConst>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
```

3 *Effects*: Equivalent to `return x.inner_ == y.inner_;`

```
template<bool OtherConst>
  requires sized_sentinel_for<zentinel<Const>, ziperator<OtherConst>>
friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
  operator-(const iterator<OtherConst>& x, const sentinel& y);

template<bool OtherConst>
  requires sized_sentinel_for<zentinel<Const>, ziperator<OtherConst>>
friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
  operator-(const sentinel& x, const iterator<OtherConst>& y);
```

4 *Effects*: Equivalent to `return x.inner_ - y.inner_;`

## § 5.7 `adjacent`

Add the following subclause to 24.7 [range.adaptors].

### § 24.7.? Adjacent view [range.adjacent]

#### § 24.7.?.1 Overview [range.adjacent.overview]

1 `adjacent_view` takes a `view` and produces a `view` whose $M^{th}$ element is a `tuple` or `pair` of the M $^{th}$ through $(M + N - 1)^{th}$ elements of the original view. If the original view has fewer than $N$ elements, the resulting view is empty.

2 The name `views::adjacent<N>` denotes a range adaptor object (24.7.2 [range.adaptor.object]). Given a subexpression `E` and a constant expression `N`, the expression `views::adjacent<N>(E)` is expression-equivalent to

.1) — `(void)E, decay-copy(views::empty<tuple<>>)` if `N` is equal to `0`,

.2) — otherwise, `adjacent_view<views::all_t<decltype((E))>, N>(E)`.

3 Define *`REPEAT(T, N)`* as a pack of *N* types, each of which denotes the same type as `T`.

§ **24.7.?.2 Class template `adjacent_view` [range.adjacent.view]**

```
namespace std::ranges {

template<forward_range V, size_t N>
  requires view<V> && (N > 0)
class adjacent_view : public view_interface<adjacent_view<V, N>>{
  V base_ = V();                     // exposition only

  template<bool> class iterator;     // exposition only
  template<bool> class sentinel;     // exposition only

  struct as_sentinel{};              // exposition only

public:
  constexpr adjacent_view() = default;
  constexpr explicit adjacent_view(V base);

  constexpr auto begin() requires (!simple-view<V>) {
    return iterator<false>(ranges::begin(base_), ranges::end(base_));
  }

  constexpr auto begin() const requires range<const V> {
    return iterator<true>(ranges::begin(base_), ranges::end(base_));
  }

  constexpr auto end() requires (!simple-view<V> && !common_range<V>) {
    return sentinel<false>(ranges::end(base_));
  }

  constexpr auto end() requires (!simple-view<V> && common_range<V>){
    return iterator<false>(as_sentinel{}, ranges::begin(base_), ranges::end(base_));
  }

  constexpr auto end() const requires range<const V> {
    return sentinel<true>(ranges::end(base_));
  }

  constexpr auto end() const requires common_range<const V> {
    return iterator<true>(as_sentinel{}, ranges::begin(base_), ranges::end(base_));
  }

  constexpr auto size() requires sized_range<V>;
  constexpr auto size() const requires sized_range<const V>;
};
```

```
}

constexpr explicit adjacent_view(V base);
```

1 *Effects*: Initializes *base_* with `std::move(base)`.

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

2 *Effects*: Equivalent to:

```
auto sz = ranges::size(base_);
sz -= std::min<decltype(sz)>(sz, N-1);
return sz;
```

§ **24.7.?.3 Class template `adjacent_view::iterator` [range.adjacent.iterator]**

```
namespace std::ranges {
  template<forward_range V, size_t N>
    requires view<V> && (N > 0)
  template<bool Const>
  class adjacent_view<V, N>::iterator {
    using Base = maybe-const<Const, V>;                                    //
        exposition only
    array<iterator_t<Base>, N> current_ = array<iterator_t<Base>, N>();    //
        exposition only
    constexpr iterator(iterator_t<Base> first, sentinel_t<Base> last);     //
        exposition only
    constexpr iterator(as_sentinel, iterator_t<Base> first, iterator_t<Base> last); //
        exposition only
  public:
    using iterator_category = input_iterator_tag;
    using iterator_concept  = see below;
    using value_type = tuple-or-pair<REPEAT(range_value_t<Base>, N)...>;
    using difference_type = range_difference_t<Base>;

    iterator() = default;
    constexpr iterator(iterator<!Const> i)
      requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

    constexpr auto operator*() const;
    constexpr iterator& operator++();
    constexpr iterator operator++(int);

    constexpr iterator& operator--() requires bidirectional_range<Base>;
    constexpr iterator operator--(int) requires bidirectional_range<Base>;

    constexpr iterator& operator+=(difference_type x)
      requires random_access_range<Base>;
    constexpr iterator& operator-=(difference_type x)
      requires random_access_range<Base>;

    constexpr auto operator[](difference_type n) const
      requires random_access_range<Base>;
```

```
    friend constexpr bool operator==(const iterator& x, const iterator& y);

    friend constexpr bool operator<(const iterator& x, const iterator& y)
      requires random_access_range<Base>;
    friend constexpr bool operator>(const iterator& x, const iterator& y)
      requires random_access_range<Base>;
    friend constexpr bool operator<=(const iterator& x, const iterator& y)
      requires random_access_range<Base>;
    friend constexpr bool operator>=(const iterator& x, const iterator& y)
      requires random_access_range<Base>;
    friend constexpr auto operator<=>(const iterator& x, const iterator& y)
      requires random_access_range<Base> &&
               three_way_comparable<iterator_t<Base>>;

    friend constexpr iterator operator+(const iterator& i, difference_type n)
      requires random_access_range<Base>;
    friend constexpr iterator operator+(difference_type n, const iterator& i)
      requires random_access_range<Base>;
    friend constexpr iterator operator-(const iterator& i, difference_type n)
      requires random_access_range<Base>;
    friend constexpr difference_type operator-(const iterator& x, const iterator& y)
      requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;

    friend constexpr auto iter_move(const iterator& i) noexcept(see below);

    friend constexpr void iter_swap(const iterator& l, const iterator& r)
      noexcept(see below)
      requires indirectly_swappable<iterator_t<Base>>;
  };
}
```

1 *iterator*::iterator_concept is defined as follows:

.1) — If V models random_access_range, then iterator_concept denotes random_access_iterator_tag.

.2) — Otherwise, if V models bidirectional_range, then iterator_concept denotes bidirectional_iterator_tag.

.3) — Otherwise, iterator_concept denotes forward_iterator_tag.

```
constexpr iterator(iterator_t<Base> first, sentinel_t<Base> last);
```

2 *Postconditions*: *current_*[0] == first is true, and for every integer *i* in [1, N), *current_*[*i*] == ranges::next(*current_*[*i*-1], 1, last) is true.

```
constexpr iterator(as_sentinel, iterator_t<Base> first, iterator_t<Base> last);
```

3 *Postconditions*: If *Base* does not model bidirectional_range, each element of *current_* is equal to last. Otherwise, *current_*[N-1] == last is true, and for every integer *i* in [0, N-1), *current_*[*i*] == ranges::prev(*current_*[*i*+1], 1, first) is true.

```
constexpr iterator(iterator<!Const> i)
  requires Const && (convertible_to<iterator_t<V>, iterator_t<Base>>;
```

4 *Effects*: Initializes each element of *current_* with the corresponding element of `i.`*current_* as an xvalue.

```
constexpr auto operator*() const;
```

5 *Effects*: Equivalent to:

```
return tuple-transform([](auto& i) -> decltype(auto) { return
      *i; }, current_);
```

```
constexpr iterator& operator++();
```

6 *Effects*: Equivalent to:

```
ranges::copy(current_ | views::drop(1), current_.begin());
++current_.back();
return *this;
```

```
constexpr iterator operator++(int);
```

7 *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
constexpr iterator& operator--() requires bidirectional_range<Base>;
```

8 *Effects*: Equivalent to:

```
ranges::copy_backward(current_ | views::take(N - 1),
      current_.end());
--current_.front();
return *this;
```

```
constexpr iterator operator--(int) requires bidirectional_range<Base>;
```

9 *Effects*: Equivalent to:

```
auto tmp = *this;
--*this;
return tmp;
```

```
constexpr iterator& operator+=(difference_type x)
  requires random_access_range<Base>;
```

10 *Effects*: Equivalent to:

```
for(auto& i : current_) { i += x; }
return *this;
```

```
constexpr iterator& operator-=(difference_type x)
  requires random_access_range<Base>;
```

11 *Effects*: Equivalent to:

```
      for(auto& i : current_) { i -= x; }
      return *this;
```

```
constexpr auto operator[](difference_type n) const
  requires random_access_range<Base>;
```

12 *Effects*: Equivalent to:

```
      return tuple-transform([&](auto& i) -> decltype(auto) { return
            i[n]; }, current_);
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y);
```

13 *Effects*: Equivalent to: `return x.current_.back() == y.current_.back();`.

```
friend constexpr bool operator<(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
```

14 *Effects*: Equivalent to: `return x.current_.back() < y.current_.back();`

```
friend constexpr bool operator>(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
```

15 *Effects*: Equivalent to: `return y < x;`

```
friend constexpr bool operator<=(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
```

16 *Effects*: Equivalent to: `return !(y < x);`

```
friend constexpr bool operator>=(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
```

17 *Effects*: Equivalent to: `return !(x < y);`

```
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
  requires random_access_range<Base> &&
           three_way_comparable<iterator_t<Base>>;
```

18 *Effects*: Equivalent to: `return x.current_.back() <=> y.current_.back();`

```
friend constexpr iterator operator+(const iterator& i, difference_type n)
  requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
  requires random_access_range<Base>;
```

19 *Effects*: Equivalent to:

```
      auto r = i;
      r += n;
      return r;
```

```
friend constexpr iterator operator-(const iterator& i, difference_type n)
  requires random_access_range<Base>;
```

20 *Effects*: Equivalent to:

```
auto r = i;
r -= n;
return r;
```

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
  requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
```

21 *Effects*: Equivalent to: `return x.current_.back() - y.current_.back();`.

```
friend constexpr auto iter_move(const iterator& i) noexcept(see below);
```

22 *Effects*: Equivalent to:

```
return tuple-transform(ranges::iter_move, i.current_);
```

23 *Remarks:* The expression within `noexcept` is equivalent to

```
noexcept(ranges::iter_move(declval<iterator_t<Base>>())) &&
is_nothrow_move_constructible_v<range_rvalue_reference_t<Base>>
```

```
friend constexpr void iter_swap(const iterator& l, const iterator& r)
  noexcept(see below)
  requires indirectly_swappable<iterator_t<Base>>;
```

24 *Preconditions:* None of the iterators in `l.current_` is equal to an iterator in `r.current_`.

25 *Effects*: For every integer *i* in `[0, N)`, performs `ranges::iter_swap(l.current_[i], r.current_[i])`.

26 *Remarks:* The expression within `noexcept` is equivalent to:

```
noexcept(ranges::iter_swap(declval<iterator_t<Base>>(),
      declval<iterator_t<Base>>()))
```

§ **24.7.?.4 Class template `adjacent_view::sentinel` [range.adjacent.sentinel]**

```
namespace std::ranges {
  namespace std::ranges {
  template<forward_range V, size_t N>
    requires view<V> && (N > 0)
  template<bool Const>
  class adjacent_view<V, N>::sentinel {
    using Base = maybe-const<Const, V>;                    // exposition only
    sentinel_t<Base> end_ = sentinel_t<Base>();            // exposition only
    constexpr explicit sentinel(sentinel_t<Base> end);     // exposition only
  public:
    sentinel() = default;
    constexpr sentinel(sentinel<!Const> i)
      requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

    template<bool OtherConst>
      requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
```

```
    friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel&
        y);

    template<bool OtherConst>
      requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst,
        V>>>
    friend constexpr range_difference_t<maybe-const<OtherConst, V>>
      operator-(const iterator<OtherConst>& x, const sentinel& y);

    template<bool OtherConst>
      requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst,
        V>>>
    friend constexpr range_difference_t<maybe-const<OtherConst, V>>
      operator-(const sentinel& y, const iterator<OtherConst>& x);
  };
}
```

```
constexpr explicit sentinel(sentinel_t<Base> end);
```

1 *Effects*: Initializes *end_* with `end`.

```
constexpr sentinel(sentinel<!Const> i)
  requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
```

2 *Effects*: Initializes *end_* with `std::move(i.end_)`.

```
template<bool OtherConst>
  requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
```

3 *Effects*: Equivalent to: `return x.current_.back() == y.end_;`.

```
template<bool OtherConst>
  requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst,
        V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
  operator-(const iterator<OtherConst>& x, const sentinel& y);
```

4 *Effects*: Equivalent to: `return x.current_.back() - y.end_;`.

```
template<bool OtherConst>
  requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst,
        V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
  operator-(const sentinel& y, const iterator<OtherConst>& x);
```

5 *Effects*: Equivalent to: `return y.end_ - x.current_.back();`.

## § 5.8 `adjacent_transform`

Add the following subclause to 24.7 [range.adaptors].

### § 24.7.? Adjacent transform view [range.adjacent.transform]

#### § 24.7.?.1 Overview [range.adjacent.transform.overview]

1 `adjacent_transform_view` takes an invocable object and a `view` and produces a `view` whose $M^{th}$ element is the result of applying the invocable object to the $M^{th}$ through $(M + N - 1)^{th}$ elements of the original view. If the original view has fewer than $N$ elements, the resulting view is empty.

2 The name `views::adjacent_transform<N>` denotes a range adaptor object (24.7.2 [range.adaptor.object]). Given subexpressions `E` and `F`,

.1) — if `N` is equal to 0, `views::adjacent_transform<N>(E, F)` is expression-equivalent to `(void)E, views::zip_transform(F)`, except that the evaluations of `E` and `F` are indeterminately sequenced.

.2) — Otherwise, the expression `views::adjacent_transform<N>(E, F)` is expression-equivalent to `adjacent_transform_view<views::all_t<decltype((E))>, decay_t<F>, N>(E, F)`.

§ **24.7.?.2 Class template `adjacent_transform_view` [range.adjacent.transform.view]**

```
namespace std::ranges {
  template<forward_range V, copy_constructible F, size_t N>
   requires view<V> && (N > 0) && is_object_v<F> &&
            regular_invocable<F&, REPEAT(range_reference_t<V>, N)...> &&
            can-reference<invoke_result_t<F&, REPEAT(range_reference_t<V>, N)...>>
  class adjacent_transform_view : public view_interface<adjacent_transform_view<V, F,
        N>> {
  semiregular-box<F> fun_;                  // exposition only
  adjacent_view<V, N> inner_;               // exposition only

  using InnerView = adjacent_view<V, N>;    // exposition only
  template<bool Const>
  using inner-iterator = iterator_t<maybe-const<Const, InnerView>>;  // exposition
        only
  template<bool Const>
  using inner-sentinel = sentinel_t<maybe-const<Const, InnerView>>;  // exposition
        only

  template<bool> class iterator;         // exposition only
  template<bool> class sentinel;         // exposition only

public:
  constexpr adjacent_transform_view() = default;

  constexpr explicit adjacent_transform_view(V base, F fun);

  constexpr auto begin() {
    return iterator<false>(*this, inner_.begin());
  }

  constexpr auto begin() const
    requires range<const InnerView> &&
             regular_invocable<const F&, REPEAT(range_reference_t<const V>, N)...>
  {
    return iterator<true>(*this, inner_.begin());
  }

  constexpr auto end() {
    return sentinel<false>(inner_.end());
```

```
  }

  constexpr auto end() requires common_range<InnerView> {
    return iterator<false>(*this, inner_.end());
  }

  constexpr auto end() const
    requires range<const InnerView> &&
             regular_invocable<const F&, REPEAT(range_reference_t<const V>, N)...>
  {
    return sentinel<true>(inner_.end());
  }

  constexpr auto end() const
    requires common_range<const InnerView> &&
             regular_invocable<const F&, REPEAT(range_reference_t<const V>, N)...>
  {
    return iterator<true>(*this, inner_.end());
  }

  constexpr auto size() requires sized_range<InnerView> {
    return inner_.size();
  }

  constexpr auto size() const requires sized_range<const InnerView> {
    return inner_.size();
  }
};

}
```

```
constexpr explicit adjacent_transform_view(V base, F fun);
```

1 *Effects*: Initializes *fun_* with `std::move(fun)` and *inner__* with `std::move(base)`.

### § 24.7.?.3 Class template `adjacent_transform_view::iterator` [range.adjacent.transform.iterator]

```
namespace std::ranges {
  template<forward_range V, copy_constructible F, size_t N>
    requires view<V> && (N > 0) && is_object_v<F> &&
             regular_invocable<F&, REPEAT(range_reference_t<V>, N)...> &&
             can-reference<invoke_result_t<F&, REPEAT(range_reference_t<V>, N)...>>
  template<bool Const>
  class adjacent_transform_view<F, V...>::iterator {
    using Parent = maybe-const<Const, adjacent_transform_view>;      // exposition
        only
    using Base = maybe-const<Const, V>;                              // exposition
        only
    Parent* parent_ = nullptr;                                       // exposition
        only
    inner-iterator<Const> inner_ = inner-iterator<Const>();          // exposition
        only

    constexpr iterator(Parent& parent, inner-iterator<Const> inner); // exposition
        only
```

```
  public:
    using iterator_category = see below;
    using iterator_concept  = typename inner-iterator<Const>::iterator_concept;
    using value_type =
      remove_cvref_t<invoke_result_t<maybe-const<Const, F>&,
        REPEAT(range_reference_t<Base>, N)...>>;
    using difference_type = range_difference_t<Base>;

    iterator() = default;
    constexpr iterator(iterator<!Const> i)
      requires Const && convertible_to<inner-iterator<false>, inner-iterator<Const>>;

    constexpr decltype(auto) operator*() const noexcept(see below);
    constexpr iterator& operator++();
    constexpr iterator operator++(int);

    constexpr iterator& operator--() requires bidirectional_range<Base>;
    constexpr iterator operator--(int) requires bidirectional_range<Base>;

    constexpr iterator& operator+=(difference_type x) requires
        random_access_range<Base>;
    constexpr iterator& operator-=(difference_type x) requires
        random_access_range<Base>;

    constexpr decltype(auto) operator[](difference_type n) const requires
        random_access_range<Base>;

    friend constexpr bool operator==(const iterator& x, const iterator& y);

    friend constexpr bool operator<(const iterator& x, const iterator& y)
      requires random_access_range<Base>;
    friend constexpr bool operator>(const iterator& x, const iterator& y)
      requires random_access_range<Base>;
    friend constexpr bool operator<=(const iterator& x, const iterator& y)
      requires random_access_range<Base>;
    friend constexpr bool operator>=(const iterator& x, const iterator& y)
      requires random_access_range<Base>;
    friend constexpr auto operator<=>(const iterator& x, const iterator& y)
      requires random_access_range<Base> && three_way_comparable<inner-
        iterator<Const>>;

    friend constexpr iterator operator+(const iterator& i, difference_type n)
      requires random_access_range<Base>;
    friend constexpr iterator operator+(difference_type n, const iterator& i)
      requires random_access_range<Base>;
    friend constexpr iterator operator-(const iterator& i, difference_type n)
      requires random_access_range<Base>;
    friend constexpr difference_type operator-(const iterator& x, const iterator& y)
      requires sized_sentinel_for<inner-iterator<Const>, inner-iterator<Const>>;
  };
}
```

1 The member *typedef-name* `iterator::iterator_category` is defined as follows:

— If `invoke_result_t<maybe-const<Const, F>&, REPEAT(range_reference_t<Base>, N)...>` is not an lvalue reference, `iterator_category` denotes `input_iterator_tag`.

— Otherwise, let `C` denote the type `iterator_traits<iterator_t<Base>>::iterator_category`.

  — If `derived_from<C, random_access_iterator_tag>` is `true`, `iterator_category` denotes `random_access_iterator_tag`;

  — Otherwise, if `derived_from<C, bidirectional_iterator_tag>` is `true`, `iterator_category` denotes `bidirectional_iterator_tag`;

  — Otherwise, if `derived_from<C, forward_iterator_tag>` is `true`, `iterator_category` denotes `forward_iterator_tag`;

  — Otherwise, `iterator_category` denotes `input_iterator_tag`.

```
constexpr iterator(Parent& parent, inner-iterator<Const> inner);
```

2 *Effects*: Initializes *parent_* with `addressof(parent)` and *inner_* with `std::move(inner)`.

```
constexpr iterator(iterator<!Const> i)
  requires Const && convertible_to<inner-iterator<false>, inner-iterator<Const>>;
```

3 *Effects*: Initializes *parent_* with `i.parent_` and *inner_* with `std::move(i.inner_)`.

```
constexpr decltype(auto) operator*() const noexcept(see below);
```

4 *Effects*: Equivalent to:

```
return apply([&](const auto&... iters) -> decltype(auto) {
  return invoke(*parent_->fun_, *iters...);
}, inner_.current_);
```

5 *Remarks:* Let `Is` be the pack `0, 1, ..., (N-1)`. The expression within `noexcept` is equivalent to `noexcept(invoke(*parent_->fun_, *get<Is>(inner_.current_)...))`.

```
constexpr iterator& operator++();
```

6 *Effects*: Equivalent to:

```
++inner_;
return *this;
```

```
constexpr iterator operator++(int);
```

7 *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
constexpr iterator& operator--() requires bidirectional_range<Base>;
```

8 *Effects*: Equivalent to:

```
--inner_;
return *this;
```

```
constexpr iterator operator--(int) requires bidirectional_range<Base>;
```

9 *Effects*: Equivalent to:

```
auto tmp = *this;
--*this;
return tmp;
```

```
constexpr iterator& operator+=(difference_type x)
  requires random_access_range<Base>;
```

10 *Effects*: Equivalent to:

```
inner_ += x;
return *this;
```

```
constexpr iterator& operator-=(difference_type x)
  requires random_access_range<Base>;
```

11 *Effects*: Equivalent to:

```
inner_ -= x;
return *this;
```

```
constexpr decltype(auto) operator[](difference_type n) const
  requires random_access_range<Base>;
```

12 *Effects*: Equivalent to:

```
return apply([&](const auto&... iters) -> decltype(auto) {
  return invoke(*parent_->fun_, iters[n]...);
}, inner_.current_);
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y);
friend constexpr bool operator<(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
  requires random_access_range<Base> && three_way_comparable<inner-iterator<Const>>;
```

13 Let *op* be the operator.

14 *Effects*: Equivalent to: `return x.inner_ op y.inner_;`

```
friend constexpr iterator operator+(const iterator& i, difference_type n)
  requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
  requires random_access_range<Base>;
```

15 *Effects*: Equivalent to: `return iterator(*i.parent_, i.inner_ + n);`

```
friend constexpr iterator operator-(const iterator& i, difference_type n)
  requires random_access_range<Base>;
```

16 *Effects*: Equivalent to: `return iterator(*i.parent_, i.inner_ - n);`

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
  requires sized_sentinel_for<inner-iterator<Const>, inner-iterator<Const>>;
```

17 *Effects*: Equivalent to: `return x.inner_ - y.inner_;`

### § 24.7.?.4 Class template `adjacent_transform_view::sentinel` [range.adjacent.transform.sentinel]

```
namespace std::ranges {
  template<forward_range V, copy_constructible F, size_t N>
    requires view<V> && (N > 0) && is_object_v<F> &&
             regular_invocable<F&, REPEAT(range_reference_t<V>, N)...> &&
             can-reference<invoke_result_t<F&, REPEAT(range_reference_t<V>, N)...>>
  template<bool Const>
  class adjacent_transform_view<V, F, N>::sentinel {
    inner-sentinel<Const> inner_;                              // exposition only
    constexpr explicit sentinel(inner-sentinel<Const> inner);  // exposition only
  public:
    sentinel() = default;
    constexpr sentinel(sentinel<!Const> i)
      requires Const && convertible_to<inner-sentinel<false>, inner-sentinel<Const>>;

    template<bool OtherConst>
      requires sentinel_for<inner-sentinel<Const>, inner-iterator<OtherConst>>
    friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel&
        y);

    template<bool OtherConst>
      requires sized_sentinel_for<inner-sentinel<Const>, inner-iterator<OtherConst>>
    friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
      operator-(const iterator<OtherConst>& x, const sentinel& y);

    template<bool OtherConst>
      requires sized_sentinel_for<inner-sentinel<Const>, inner-iterator<OtherConst>>
    friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
      operator-(const sentinel& x, const iterator<OtherConst>& y);
  };
}
```

```
constexpr explicit sentinel(inner-sentinel<Const> inner);
```

1 *Effects*: Initializes *inner_* with `inner`.

```
constexpr sentinel(sentinel<!Const> i)
  requires Const && convertible_to<inner-sentinel<false>, inner-sentinel<Const>>;
```

2 *Effects*: Initializes *inner_* with `std::move(i.inner_)`.

```
template<bool OtherConst>
  requires sentinel_for<inner-sentinel<Const>, inner-iterator<OtherConst>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
```

3 *Effects*: Equivalent to `return x.inner_ == y.inner_;`

```
template<bool OtherConst>
  requires sized_sentinel_for<inner-sentinel<Const>, inner-iterator<OtherConst>>
friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
  operator-(const iterator<OtherConst>& x, const sentinel& y);

template<bool OtherConst>
  requires sized_sentinel_for<inner-sentinel<Const>, inner-iterator<OtherConst>>
friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
  operator-(const sentinel& x, const iterator<OtherConst>& y);
```

4 *Effects*: Equivalent to `return x.inner_ - y.inner_;`

## § 5.9 Feature-test macro

Add the following macro definition to 17.3.2 [version.syn], header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_ranges_zip 20XXXXL // also in <ranges>, <tuple>, <utility>
```

# § 6 Acknowledgements

Thanks to Barry Revzin for implementing this entire paper from spec and finding several wording issues in the process.

# § 7 References

[N4878] Thomas Köppe. 2020-12-15. Working Draft, Standard for Programming Language C++.
https://wg21.link/n4878

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges.
https://wg21.link/p2214r0

[range-v3.1592] kitegi. 2020. `zip` does not satisfy the semantic requirements of `bidirectional_iterator` when the ranges have different lengths.
https://github.com/ericniebler/range-v3/issues/1592

[range-v3.704] Eric Niebler. 2017. Demand-driven view strength weakening.
https://github.com/ericniebler/range-v3/issues/704